Google Cloud Security Command Center — Posture & Threat Management

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Reference design . . . 5

Benefits . . . 6

Limitations & boundaries . . . 6

Deployment model . . . 6

Requirements 7

1. A confirmed SCC edition (and knowledge of what it includes) . . . 7

2. SCC administrative IAM roles (least privilege) . . . 7

3. Required APIs enabled . . . 8

4. A SecOps project for exports and notifications . . . 8

5. A SIEM / notification destination and its ingestion contract . . . 9

6. A finding-triage operating model (ownership + SLAs) . . . 9

7. A named pilot scope . . . 9

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions 10

1. A Google Cloud Organization linked to Cloud Identity / Workspace . . . 10

2. Organization-level administrative access . . . 10

3. A Cloud Billing account . . . 11

4. The gcloud / IaC toolchain and API enablement . . . 11

5. A resource-hierarchy, naming & environment design . . . 11

6. Security, network & governance stakeholders and policy . . . 12

Implementation 12

Stage 1 — Assess (read-only) . . . 12

Stage 2 — Design . . . 13

Stage 3 — Configure (dry-run first, then apply) . . . 14

Stage 4 — Pilot (validate on Pilot Scope – Prod-Ready – v1) . . . 15

Stage 5 — Validate . . . 15

Stage 6 — Expand (org-wide) . . . 15

Testing & Validation 16

Test 1 — SCC edition and services are as designed . . . 16

Test 2 — Notification config publishes findings . . . 16

Test 3 — SIEM / downstream ingestion works . . . 16

Test 4 — Posture detector fires on a real misconfiguration . . . 17

Test 5 — Mute rules behave as designed . . . 17
Test 6 — Finding remediation closes the finding . . . 17
Test 7 — Triage & SLA report reflects reality . . . 17
Test 8 — Least-privilege access holds . . . 18
Test 9 — Rollback works . . . 18
Troubleshooting 18
No findings appear for a project or folder . . . 18
The `gcloud scc ...` command is not recognized / flags differ . . . 18
Notification config created but no messages on the topic . . . 19
Messages on the topic but the SIEM isn't ingesting . . . 19
Permission denied creating configs / listing findings . . . 19
The finding queue is overwhelming (too much noise) . . . 19
A remediated finding stays ACTIVE . . . 20
Triage/SLA report shows wrong owners or SLA math . . . 20
`dry-run` looked fine but `--apply` failed . . . 20
Rollback needed after an org-wide change caused a noise spike or issue . . . 20
Best Practices 20
Coverage & edition . . . 20
Noise control (so responders trust the queue) . . . 21
Triage & ownership . . . 21
Notifications & export . . . 21
Security & safety . . . 21
Operating rhythm . . . 22
Deliverables Summary 22
Documentation . . . 22
Python tools (5) . . . 23
Packaged artifacts . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary

Outcome. Google Cloud risk becomes **continuously visible and actually worked**. Misconfigurations, vulnerabilities, and threats across the Organization are surfaced as normalized findings in **Security Command Center (SCC)**, routed to owners, exported to the SIEM, and remediated within severity-based SLAs — from one security console instead of scattered, ignored alerts.

The problem. Without a central security view, Google Cloud misconfigurations and threats go unseen until they become an incident. Even where SCC is switched on, findings often pile up with no owner, no SLA, and no path into the SecOps tooling the rest of the estate already uses.

The approach. This engagement operationalizes SCC end-to-end: enable and tune it at the Organization (a deliberate **posture baseline** with documented mute rules to control noise), wire **Pub/Sub notifications** and a **SIEM export** so findings flow into existing detection-and-response pipelines, and stand up a **triage operating model** — ownership by scope plus a severity→SLA table — so findings are worked, not just watched. Five Python tools (`gcloud` / Cloud Client Libraries) assess coverage, set up the exports, generate an SLA triage report, check posture guardrails, and validate scope/config.

Why it is safe and governed. Delivery follows a pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback before any org-wide change. Every configuration tool is **read-only or dry-run by default**, with **no hardcoded credentials or committed keys** (ADC / environment only), and SCC access is least-privilege (admins configure; triagers edit finding state). Because SCC **editions and detectors are version-dependent**, the guidance explicitly instructs the team to **verify the exact edition and included detectors** rather than assuming a capability exists.

Bottom line. Google Cloud risk continuously visible and worked — misconfigurations and threats surfaced, routed to owners, and remediated within SLA — from one security console.

Solution Overview

What this is

Security Command Center (SCC) is Google Cloud's built-in security and risk-management platform. It scans the resource hierarchy for **misconfigurations and vulnerabilities** (posture findings), surfaces **active threats** from log- and behavior-based detectors, and presents everything as normalized **findings** in one console. This solution takes SCC from "turned on somewhere" to **operationalized at the Organization**: enabled org-wide, tuned to cut noise, wired to **Pub/Sub notifications** and a **SIEM export**, and backed by a **triage / ownership / remediation-SLA workflow** so findings are not just seen but worked.

The engagement delivers the enablement design, the exact `gcloud`/Console procedure, a triage operating model, and **five Python tools** (`gcloud` / Cloud Client Library automation, read-only or dry-run by default) that assess coverage, set up exports, generate an SLA triage report, check posture guardrails, and manage notification/export configuration.

> ***Editions & detectors are version-dependent — verify your edition.*** *SCC is offered in tiers/editions whose exact names, packaging, and included detectors* ***change over time*** *(historically Standard / Premium, and the* ***Security Command Center Enterprise*** *tier). Which* ***posture services*** *(e.g.*

*Security Health Analytics), **vulnerability** scanners, and **threat detectors** (e.g. Event Threat Detection, Container Threat Detection, Virtual Machine Threat Detection) are available — and whether a feature is GA or preview — **depends on your active edition and can change**. This solution names detectors and services **as categories** and instructs you to **confirm the exact set in your Organization** against Google's current SCC documentation before relying on any single one. Do not assume a detector exists because it is referenced here.*

How it works

SCC operates at or below the **Organization** node and aggregates findings from its enabled **services**:

1. **Posture / misconfiguration** — a health-analytics service continuously evaluates resources

against a catalog of detectors (public buckets, open firewall rules, over-broad IAM, disabled logging, weak key rotation, etc.) and raises a finding per violation, each with a severity and a category.

1. **Vulnerabilities** — scanners identify software/package vulnerabilities and exposed

surfaces on in-scope workloads (availability depends on edition).

1. **Threat detection** — log- and behavior-based detectors raise findings on suspicious

activity (anomalous IAM grants, crypto-mining signals, malware, data-exfiltration patterns, and similar) — the exact detector set is **edition-dependent**.

1. **Notifications & export** — SCC publishes findings to a **Pub/Sub topic** via a

notification config; downstream subscribers (a SIEM connector, a ticketing webhook, a function) act on them. Continuous export to a **SIEM** (via the Pub/Sub topic or the SCC API) gives SecOps a single stream.

1. **Triage & remediation** — findings are routed to an **owner** by project/service/category,

worked to a **remediation SLA** keyed to severity, and their state (`ACTIVE` → `INACTIVE`) plus mute rules drive the metrics that prove risk is being reduced.

Reference design

The reference design in this guide uses these named artifacts (reused verbatim in the steps, tests, and scripts):

• **SCC Enablement – Org – Pilot** — the org-level activation and service configuration validated

first against a pilot scope.

• **Finding Notification – Org – v1** — the SCC notification config that publishes findings to

Pub/Sub.

• **Finding Export Topic – SecOps – v1** — the Pub/Sub topic (+ subscription) that carries

findings to the SIEM / downstream consumers.

• **Posture Baseline – Corp – v1** — the agreed set of enabled posture detectors and their

mute/exception rules (the "known-good" posture).

• **Triage Runbook – SecOps – v1** — the ownership matrix, severity→SLA table, and triage cycle.

• **Pilot Scope – Prod-Ready – v1** — the folder/project subset the whole configuration is

validated on before org-wide expansion.

Benefits

• **One console for cloud risk** — posture, vulnerabilities, and threats in a single normalized finding stream instead of scattered, per-service alerts.

• **Findings that get worked, not just seen** — explicit ownership and severity-based SLAs turn a dashboard into an operating process with measurable closure.

• **SIEM-native** — findings flow to existing SecOps tooling via Pub/Sub, so cloud risk joins the same detection-and-response pipeline as the rest of the estate.

• **Noise controlled** — a deliberate posture baseline plus mute rules keep the signal high so responders trust the queue.

• **Safe to adopt** — pilot-first, with pass/fail gates and a documented rollback for every org-wide change.

Limitations & boundaries

• **Edition-gated capability.** The available services and detectors depend on your **SCC edition** — verify the exact set; this solution does not guarantee any specific detector is present in your tenant.

• **Detection ≠ prevention.** SCC surfaces and routes risk; it does not by itself remediate. Remediation is performed by the owning team (optionally with SCC/Cloud Functions automation you build separately).

• **Scope is Google Cloud.** SCC covers the Google Cloud Organization and its resources; other clouds/estates are out of scope (some editions offer multi-cloud connectors — **verify**).

• **Legal/compliance interpretation deferred.** Whether a finding constitutes a reportable compliance gap is a determination for the organization's security/compliance/legal functions; this solution provides the technical detection, routing, and evidence.

• **IaC at the pattern level.** Terraform / Cloud Foundation Toolkit blueprints are shown as a pattern with representative snippets; module internals defer to the provider's own docs.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback before any org-wide change:

1. **Assess** — inventory current SCC state, edition, org access, and SIEM/notification targets (read-only; script #1).

1. **Design** — decide the posture baseline, notification/export topology, ownership matrix, and

severity→SLA table; pick the pilot scope.

1. **Configure** — enable/tune SCC services, create the notification config and export topic,

apply the posture baseline (dry-run first; scripts #2, #5).

1. **Pilot** — run the configuration against **Pilot Scope – Prod-Ready – v1**; confirm findings

surface, route, and export end-to-end.

1. **Validate** — run a full triage cycle; confirm findings are owned and closed within SLA

(scripts #3, #4) against the pass/fail gates.

1. **Expand** — extend org-wide once the gates pass; keep the triage runbook and metrics running.

Requirements

These are the **solution-specific** prerequisites for operationalizing Security Command Center (SCC). They build on the shared **Global Requirements — Google Cloud Platform (GCP)** (`global-requirements-gcp.md`): a Google Cloud **Organization** linked to Cloud Identity / Workspace, Organization-level administrative access, an active Cloud **Billing** account, the `gcloud` / Python toolchain with API-enablement rights, a resource-hierarchy & naming design, and engaged security/network/governance stakeholders. Verify those first; the items below are what SCC additionally needs.

Each prerequisite lists **Description / Why required / How to verify / How to implement if missing**.

1. A confirmed SCC edition (and knowledge of what it includes)

• **Description:** An **active Security Command Center edition/tier** on the Organization, and a

documented list of which **posture services, vulnerability scanners, and threat detectors** that edition actually includes.

• **Why required:** SCC capability is **edition-gated and version-dependent** — the detectors and

services this solution tunes must exist in your edition. Configuring a service you do not have wastes cycles and creates false confidence.

• **How to verify:** In the Console, open **Security Command Center** → **Settings** and note the

active edition and enabled services; cross-check against Google's **current** SCC editions documentation (`cloud.google.com/security-command-center/docs`). With the API: `gcloud scc settings services describe --organization=ORG_ID` (verify exact command/flags for your edition — API surface varies by tier). **Do not assume a detector exists** because it is named in this guide.

• **How to implement if missing:** Activate or upgrade the desired SCC edition at the Organization

(an Org-admin / billing-admin action in **Security Command Center** → **Settings**), then re-inventory the included services. Where an edition change alters billing, route it through the billing/FinOps owner.

2. SCC administrative IAM roles (least privilege)

• **Description:** The identities operating SCC hold the appropriate **SCC roles at the

Organization** — e.g. `roles/securitycenter.admin` for configuration, and read/finding-editor roles (`roles/securitycenter.adminViewer`, `roles/securitycenter.findingsEditor`) for triagers — rather than blanket Owner.

• **Why required:** Enabling services, creating notification configs, and muting findings are

privileged org-level operations; triagers need to edit finding state without broad rights. Least-privilege limits blast radius.

• **How to verify:** `gcloud organizations get-iam-policy ORG_ID` and confirm the SCC roles are

bound to the intended groups (not individuals). Confirm the exact role names against current SCC IAM docs — role catalogs evolve.

• **How to implement if missing:** Have an Organization Administrator grant the SCC roles to

purpose-named groups under change control (e.g. a `grp-scc-admins` and `grp-scc-triage` group).

3. Required APIs enabled

• **Description:** The APIs SCC and this solution depend on are **enabled** on the appropriate

project(s): the **Security Command Center API** (`securitycenter.googleapis.com`), **Pub/Sub** (`pubsub.googleapis.com`), and, if used, **Cloud Functions / Cloud Run** and **BigQuery** for downstream processing/export.

• **Why required:** The notification config, export topic, and automation call these APIs; a

disabled API blocks the corresponding step.

• **How to verify:** `gcloud services list --enabled --project=PROJECT_ID` and confirm the API IDs

above are present.

• **How to implement if missing:** `gcloud services enable securitycenter.googleapis.com

pubsub.googleapis.com --project=PROJECT_ID` (add functions/bigquery as needed).

4. A SecOps project for exports and notifications

• **Description:** A dedicated **project to host the Pub/Sub export topic**, subscription(s), and

any export/processing resources — kept separate from workload projects (the **Finding Export Topic – SecOps – v1** lives here).

• **Why required:** Centralizing the export/notification plumbing in one governed project keeps IAM

tight, avoids coupling security tooling to workload lifecycles, and gives SecOps clear ownership.

• **How to verify:** Confirm a SecOps/security project exists in the hierarchy, linked to billing,

with the SecOps group as owner.

• **How to implement if missing:** Create the project under the security/shared folder, link

billing, enable the APIs from §3, and grant the SecOps group appropriate roles.

5. A SIEM / notification destination and its ingestion contract

• **Description:** A confirmed **downstream destination** for findings — a SIEM (via a Pub/Sub pull/push subscription or the SIEM's Google Cloud connector), a ticketing webhook, and/or BigQuery — with the **ingestion method, credentials model, and field mapping** agreed with the SecOps/SIEM owner.

• **Why required:** The notification/export config is only useful if something consumes it; the push/pull method and the SIEM's expected schema must be known before wiring.

• **How to verify:** Confirm with the SIEM owner the ingestion mechanism (pull subscription, push endpoint, or vendor connector) and that a service account / credential path exists for it. Verify the SIEM's SCC/Pub/Sub connector requirements against the **SIEM vendor's current docs** (these change).

• **How to implement if missing:** Stand up the subscription/connector with the SIEM owner; provision a least-privilege service account for the consumer (no exported keys committed to disk — use workload identity / ADC where possible).

6. A finding-triage operating model (ownership + SLAs)

• **Description:** An agreed **ownership matrix** (which team owns findings by project / service / category) and a **severity** → **remediation-SLA table** (e.g. Critical / High / Medium / Low target times), captured as **Triage Runbook – SecOps – v1**.

• **Why required:** Findings only reduce risk if someone owns them and works them to a deadline; without an operating model, SCC becomes an ignored dashboard.

• **How to verify:** Confirm the ownership matrix and SLA table are documented and signed off by the security and application owners.

• **How to implement if missing:** Facilitate a working session with SecOps and application owners to define ownership by scope and set severity-based SLAs; record them in the runbook (this guide's Implementation section produces it).

7. A named pilot scope

• **Description:** A specific **folder or set of projects** — **Pilot Scope – Prod-Ready – v1** — on which the SCC configuration, notifications, export, and triage cycle are validated before org-wide expansion.

• **Why required:** The house deployment model is pilot-first with pass/fail gates; a bounded pilot proves the end-to-end flow and lets you tune noise before it hits every team.

• **How to verify:** Confirm the pilot folder/projects with the platform owners; ensure they contain realistic-enough resources to generate representative findings.

• **How to implement if missing:** Select a representative non-critical folder/project set and record

it as the pilot scope; if none exists, the Landing Zone solution defines the hierarchy first.

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions

These requirements apply to the **Google Cloud Platform (GCP)** infrastructure and zero-trust solutions in the **Google** category (landing zone, Cloud IAM, networking / VPC Service Controls, Security Command Center, FinOps, and BeyondCorp Enterprise). They are the common foundation each GCP solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every GCP solution).*** *The **Google Cloud Console** (console.cloud.google.com), the `gcloud` **CLI**, and **Cloud Identity / Google Admin** are authored at exact, current click-by-click / command detail from Google's public documentation. Where a capability, API, or default is **version- or preview-dependent** (new SCC tiers, BeyondCorp features, org-policy constraints, service-perimeter behaviors), it is **cited or flagged "verify in your organization"** — never invented. **Infrastructure-as-Code** (Terraform, the Cloud Foundation Toolkit / Terraform blueprints, Config Controller) is used at the **pattern level** — the solutions give the design, the `gcloud`/console procedure, and representative IaC, and defer provider/module internals to the tool's own docs. Automation is delivered as **Python** (Cloud Client Libraries / `gcloud` wrappers), never with hardcoded credentials or keys.*

1. A Google Cloud Organization linked to Cloud Identity / Workspace

• **Description:** A **GCP Organization** resource (the root of the resource hierarchy), which exists once the domain's **Cloud Identity** or **Google Workspace** account is set up. Solutions build the folder/project hierarchy beneath it.

• **Why required:** Org-level policy, IAM, VPC Service Controls, and SCC all operate at or below the Organization node; without it there is no place to anchor centralized governance.

• **How to verify:** In the Cloud Console resource selector (or `gcloud organizations list`), confirm an Organization resource exists and its ID.

• **How to implement if missing:** Establish Cloud Identity/Workspace for the domain so the Organization is created, then verify domain ownership.

2. Organization-level administrative access

• **Description:** An account (ideally a **break-glass super-admin** plus a delegated **Organization Administrator** / least-privilege roles the solution names) able to set org policies, create folders/projects, and grant IAM at the Organization node.

• **Why required:** Foundational changes (resource hierarchy, org policies, IAM bindings, VPC-SC perimeters) require Organization-level authority.

• **How to verify:** Confirm the account holds `roles/resourcemanager.organizationAdmin` (and the

specific roles each solution lists) via `gcloud organizations get-iam-policy`.

• **How to implement if missing:** Have a super-admin grant the roles under change control; keep

break-glass accounts audited and MFA-protected.

3. A Cloud Billing account

• **Description:** An active **Cloud Billing account** linked (or linkable) to the projects the

solution provisions, with a billing administrator available.

• **Why required:** Projects that host resources (networks, SCC, workloads) must be linked to billing;

FinOps/budget work operates directly on the billing account.

• **How to verify:** `gcloud billing accounts list` shows an OPEN account; confirm billing-admin access.

• **How to implement if missing:** Create/associate a billing account (Google or reseller) before

provisioning billable resources.

4. The gcloud / IaC toolchain and API enablement

• **Description:** A workstation or pipeline with the **Google Cloud SDK (**`gcloud`**,** `gsutil`**)**,

Python 3.10+ with the **Cloud Client Libraries**, and (where used) **Terraform**; plus the ability to **enable the required Google Cloud APIs** on target projects.

• **Why required:** Configuration, automation, and the provided Python tools run through these; most

services require their API to be enabled first.

• **How to verify:** `gcloud version`, `python --version`, and `gcloud services list --enabled` on a

target project.

• **How to implement if missing:** Install the SDK + libraries; enable the named APIs

(`gcloud services enable …`) before running the steps.

5. A resource-hierarchy, naming & environment design

• **Description:** An agreed **folder/project structure** (e.g. by department/environment), a

naming/labeling convention, and the **environments** in scope (prod / non-prod / shared), plus a **pilot project or folder** to validate on before org-wide rollout.

• **Why required:** IAM, org policy, networking, and cost allocation all key off the hierarchy and

labels; every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

• **How to verify:** Confirm the intended hierarchy, naming convention, and a named pilot scope with

the platform owners.

• **How to implement if missing:** Define the hierarchy and naming standard (this is often the first solution — the Landing Zone) before building dependent controls.

6. Security, network & governance stakeholders and policy

• **Description:** Engagement with the **security (CISO/SecOps), network, and cloud-governance / FinOps** stakeholders whose policies the configuration must satisfy (least-privilege model, allowed regions, data-residency, network egress, threat-response ownership, budget/showback rules).

• **Why required:** Org policies, IAM boundaries, perimeter design, and cost governance must align to enterprise security and financial policy, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named approvers/owners for security, networking, and cost.

• **How to implement if missing:** Identify the owners and capture the policy requirements before applying org-wide guardrails.

*The deployment model across every GCP solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for org-wide changes. Automation is delivered as **Python** (`gcloud` / Cloud Client Libraries) — never with hardcoded credentials, exported service-account keys committed to disk, or secrets in code.*

Implementation

Follow these steps in order. They implement the reference design using the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with pass/fail gates and a rollback for every org-wide change. Every artifact is **named** with the `<Purpose> – <Scope> – <Stage>` convention and reused verbatim in later steps, testing, and scripts.

> ***Verify your edition throughout.*** *Command surfaces, service IDs, and detector names below are **edition- and version-dependent**. Before running any step, confirm the exact `gcloud scc` subcommands, service identifiers, and detector names against Google's **current** SCC docs and your active edition. Where a command flag differs in your tenant, use the one your `--help` reports — do not assume.*

Placeholders: `ORG_ID` (organization numeric ID), `SECOPS_PROJECT` (the SecOps project from Requirements §4), `PILOT_FOLDER` (the pilot folder/project set).

Stage 1 — Assess (read-only)

1.1 Confirm org, edition, and access

• Confirm the Organization: `gcloud organizations list` → record `ORG_ID`.

• Open **Security Command Center** → **Settings** and record the **active edition** and the list of

enabled services (posture / vulnerability / threat). Cross-check the included detectors against Google's current SCC editions documentation.

- Confirm your account holds `roles/securitycenter.admin` at the org (Requirements §2): `gcloud organizations get-iam-policy ORG_ID`.

1.2 Inventory current SCC coverage

- Run the assessment tool (read-only) to snapshot enabled services, existing notification configs, and finding volume by severity/category:

```
python scripts/scc_coverage_assessment.py --organization ORG_ID --out reports/scc-coverage.json
```

- **Gate A (Assess):** you have a documented edition, enabled-service list, current finding baseline, and confirmed admin access. If the edition lacks a service the design assumes, resolve (upgrade or drop that service from scope) before proceeding.

Stage 2 — Design

2.1 Define the posture baseline

- From the assessment output, agree the enabled **posture detectors** and their intended mute/exception rules. Record this as **Posture Baseline – Corp – v1** (the enabled detectors + documented exceptions with justification and owner).

2.2 Design notifications & export topology

- Decide the destinations (SIEM via Pub/Sub pull/push, ticketing webhook, BigQuery) per Requirements §5. The plumbing:
- **Finding Export Topic – SecOps – v1** — the Pub/Sub topic in `SECOPS_PROJECT`.
- **Finding Notification – Org – v1** — the SCC notification config that publishes to that topic.

2.3 Define the triage operating model

- Produce **Triage Runbook – SecOps – v1**: the **ownership matrix** (owner by project/service/category) and the **severity → SLA table**, e.g.:

Severity	Target acknowledgement	Target remediation SLA
Critical	1 hour	24 hours
High	4 hours	3 business days
Medium	1 business day	14 days
Low	3 business days	30 days (or accept)

(Adjust to the organization's risk policy — this table is a starting point, not a mandate.)

- **Gate B (Design):** posture baseline, export topology, and triage runbook are documented and signed off by SecOps + application owners; **Pilot Scope – Prod-Ready – v1** is named.

Stage 3 — Configure (dry-run first, then apply)

> *All write steps are **dry-run by default** in the provided tools. Review the dry-run plan, then re-run with `--apply` under change control.*

3.1 Enable required APIs

```
gcloud services enable securitycenter.googleapis.com pubsub.googleapis.com \
  --project=SECOPS_PROJECT
```

(Add `cloudfunctions.googleapis.com` / `bigquery.googleapis.com` if used.)

3.2 Create the export topic — Finding Export Topic – SecOps – v1

```
gcloud pubsub topics create finding-export-secops-v1 --project=SECOPS_PROJECT
gcloud pubsub subscriptions create finding-export-secops-v1-sub \
  --topic=finding-export-secops-v1 --project=SECOPS_PROJECT
```

Grant the SCC notification service account publish rights on the topic (SCC uses a per-org service account of the form `service-org-ORG_ID@gcp-sa-scc-notification.iam.gserviceaccount.com` — **verify the exact service-account address for your org** in the Console before binding):

```
gcloud pubsub topics add-iam-policy-binding finding-export-secops-v1 \
  --project=SECOPS_PROJECT \
  --member="serviceAccount:service-org-ORG_ID@gcp-sa-scc-notification.iam.gserviceaccount.com" \
  --role="roles/pubsub.publisher"
```

3.3 Create the notification config — Finding Notification – Org – v1

Publish active findings to the topic. Use the config helper (dry-run first):

```
python scripts/scc_notify_export_setup.py \
  --organization ORG_ID --project SECOPS_PROJECT \
  --topic finding-export-secops-v1 \
  --config-id finding-notification-org-v1 \
  --filter 'state="ACTIVE"'                # tune the filter to your baseline
  # dry-run by default; add --apply to create
```

This wraps `gcloud scc notifications create` (verify the exact subcommand/flags for your edition). Scope the `--filter` to the categories/severities you want streamed (e.g. exclude muted or low-severity noise at the export edge if the SIEM prefers).

3.4 Apply the posture baseline — Posture Baseline – Corp – v1

- Enable/disable posture detectors to match the baseline and apply the agreed **mute rules** for

documented exceptions. Check drift first (read-only), then apply mutes with the guardrail tool:

```
python scripts/scc_posture_check.py --organization ORG_ID \
  --baseline config/posture-baseline-corp-v1.yaml --out reports/posture-drift.json
```

Create mute rules for accepted exceptions (dry-run first):

```
python scripts/scc_notify_export_setup.py --organization ORG_ID \
  --mute-config config/posture-mutes-corp-v1.yaml   # add --apply to create mute rules
```

(Wraps SCC mute-config APIs — verify subcommands for your edition.)

- **Gate C (Configure):** notification config exists and publishes to the topic; the subscription

receives a test finding; the posture baseline + mutes are applied; no unintended detector was disabled.

Stage 4 — Pilot (validate on Pilot Scope – Prod-Ready – v1)

4.1 Generate representative findings in the pilot scope

• Confirm SCC is surfacing findings for `PILOT_FOLDER`. If needed, create a **deliberately misconfigured, non-sensitive test resource **in a pilot project (e.g. a firewall rule opening a port to `0.0.0.0/0` on a throwaway VPC) to prove the posture detector fires.** Remove it after the test.**

4.2 Confirm end-to-end flow

• Verify the finding: appears in the Console, is **published to the topic**, and is **received by the SIEM/subscription**. Pull a message to confirm delivery:

```
gcloud pubsub subscriptions pull finding-export-secops-v1-sub \
  --project=SECOPS_PROJECT --auto-ack --limit=5
```

4.3 Run one triage cycle

• Assign the finding to its owner per **Triage Runbook – SecOps – v1**, remediate the test misconfiguration, and confirm SCC marks the finding `INACTIVE` on the next scan.

• Generate the SLA/triage report:

```
python scripts/scc_triage_sla_report.py --organization ORG_ID \
  --runbook config/triage-runbook-secops-v1.yaml --out reports/triage-sla.html
```

• **Gate D (Pilot):** a pilot finding was surfaced → routed → exported → owned → remediated → closed within SLA, end-to-end. If any leg fails, fix and re-run the pilot; **do not expand**.

Stage 5 — Validate

• Run the full validation suite in `05-testing.md` against the pilot scope. Confirm every gate passes: coverage as designed, notifications delivered, export ingested by the SIEM, posture baseline enforced, and the triage report shows findings owned with SLA tracking.

Stage 6 — Expand (org-wide)

• Once all gates pass, extend scope from `PILOT_FOLDER` to the Organization: the notification config is already org-scoped, so expansion is primarily (a) confirming all folders/projects are in SCC scope, (b) applying the posture baseline org-wide, and (c) onboarding remaining owners to the triage runbook.

• Communicate the change, then monitor finding volume for a noise spike; tune mutes as needed.

Rollback (documented, for any org-wide change)

• **Notifications/export:** delete or disable the notification config to stop the stream — `gcloud scc notifications delete finding-notification-org-v1 --organization=ORG_ID` (verify subcommand); findings remain in SCC, only the export stops.

• **Posture mutes:** mute rules are reversible — delete the mute config to un-mute; no finding data

is lost.

- **Service enablement:** disabling an SCC service stops new findings from it but is a

configuration change, not data loss — re-enable to resume. Record every change so it can be reverted cleanly.

- **Test resources:** ensure any deliberately misconfigured pilot test resource (§4.1) is deleted.

Testing & Validation

Run these procedures against **Pilot Scope – Prod-Ready – v1** first (Stage 4/5), then re-confirm after org-wide expansion. Each test states the procedure and an explicit **pass/fail** signal. These map to the gates in `03-implementation.md`.

> ***Verify your edition.*** *Service and detector names below are edition-dependent; confirm the exact identifiers in your tenant before asserting a test's expected result.*

Test 1 — SCC edition and services are as designed

- **Procedure:** In **Security Command Center** → **Settings**, confirm the active edition and that the

posture / vulnerability / threat services in **Posture Baseline – Corp – v1** are enabled. Cross-run: `python scripts/scc_coverage_assessment.py --organization ORG_ID --out reports/scc-coverage.json`.

- **Pass:** every service the design assumes is enabled and included in the edition.
- **Fail:** a designed service is missing or not in the edition → resolve (upgrade or de-scope)

before continuing.

Test 2 — Notification config publishes findings

- **Procedure:** Confirm **Finding Notification – Org – v1** exists and targets

Finding Export Topic – SecOps – v1: `gcloud scc notifications describe finding-notification-org-v1 --organization=ORG_ID` (verify subcommand). Create/allow a test finding in the pilot scope (see Test 4), then pull: `gcloud pubsub subscriptions pull finding-export-secops-v1-sub --project=SECOPS_PROJECT --auto-ack --limit=5`.

- **Pass:** the finding is delivered as a Pub/Sub message with the expected finding JSON.
- **Fail:** no message → check the config filter, the topic IAM binding for the SCC notification

service account, and that the finding matches the config's `--filter`.

Test 3 — SIEM / downstream ingestion works

- **Procedure:** Confirm the SIEM (or downstream consumer) received and parsed the same finding via

its subscription/connector. Check the SIEM's search UI or the consumer's logs.

- **Pass:** the finding appears in the SIEM with fields mapped per the ingestion contract

(Requirements §5).

- **Fail:** message published to Pub/Sub but absent in SIEM → check the subscription/connector, consumer service-account permissions, and field mapping.

Test 4 — Posture detector fires on a real misconfiguration

- **Procedure:** In a **throwaway pilot project**, create a deliberately misconfigured, **non-sensitive** resource (e.g. a firewall rule opening a port to `0.0.0.0/0` on a test VPC). Wait for the next posture scan.
- **Pass:** SCC raises the corresponding posture finding (correct category + severity) within the scan window. **Delete the test resource afterward.**
- **Fail:** no finding after the scan window → confirm the posture service is enabled for that project and not muted by **Posture Baseline – Corp – v1**.

Test 5 — Mute rules behave as designed

- **Procedure:** For one documented exception in the baseline, confirm its mute rule suppresses the matching finding (finding shows as muted, not surfaced to the export filter if the filter excludes muted). Run: `python scripts/scc_posture_check.py --organization ORG_ID --baseline config/posture-baseline-corp-v1.yaml --out reports/posture-drift.json`.
- **Pass:** intended exceptions are muted; nothing outside the exception list is muted (no drift).
- **Fail:** an un-approved finding is muted, or an approved exception is not → correct the mute config.

Test 6 — Finding remediation closes the finding

- **Procedure:** Remediate the misconfiguration from Test 4 (delete the offending firewall rule). Wait for the next scan.
- **Pass:** SCC transitions the finding to `INACTIVE` (remediated) on rescan.
- **Fail:** finding stays `ACTIVE` after rescan → confirm the resource was actually fixed and the scan completed.

Test 7 — Triage & SLA report reflects reality

- **Procedure:** Run `python scripts/scc_triage_sla_report.py --organization ORG_ID --runbook config/triage-runbook-secops-v1.yaml --out reports/triage-sla.html` and review.
- **Pass:** the report lists active findings with correct severity, computed SLA due dates, owner (per the runbook matrix), and an at-risk/breached flag; the closed test finding is excluded from the active queue.
- **Fail:** wrong owner mapping, missing findings, or incorrect SLA math → fix the runbook mapping

or the tool inputs and re-run.

Test 8 — Least-privilege access holds

- **Procedure:** Confirm SCC roles are bound to the intended groups (not blanket Owner) and that a triager can edit finding state but cannot change org policy: `gcloud organizations get-iam-policy ORG_ID`.
- **Pass:** admins hold `securitycenter.admin`, triagers hold finding-editor/viewer roles, no unnecessary broad grants.
- **Fail:** over-broad grants present → remediate with the Organization Administrator.

Test 9 — Rollback works

- **Procedure:** In a controlled test, disable **Finding Notification – Org – v1** and confirm the export stream stops without losing findings in SCC; re-enable and confirm the stream resumes.
- **Pass:** export stops/starts cleanly on config change; finding data in SCC is unaffected.
- **Fail:** findings lost or stream cannot be restored → investigate before relying on the rollback.

Overall exit criteria (all must pass to Expand): Tests 1–4 and 6–7 pass on the pilot scope (coverage as designed, notifications delivered, SIEM ingesting, detector fires, remediation closes, triage/SLA report correct), Test 8 confirms least privilege, and Test 9 confirms a clean rollback.

Troubleshooting

Common failure modes when operationalizing Security Command Center (SCC), with resolutions. Names in **bold** are the reference-design artifacts.

> ***Verify your edition first.*** *Many "it's not working" symptoms are actually "that service/detector isn't in my edition." Before deep debugging, confirm the active edition and enabled services in* ***Security Command Center*** *→* ***Settings****, and confirm the exact* `gcloud scc` *subcommands with* `--help` *— the API surface varies by edition/version.*

No findings appear for a project or folder

- **Likely cause:** the posture/threat **service isn't enabled** for that scope, the project is out of SCC scope, or the finding is **muted** by **Posture Baseline – Corp – v1**.
- **Resolve:** Confirm the service is enabled in Settings for that scope; confirm the project is in the SCC-covered hierarchy; run `scripts/scc_posture_check.py` to spot an un-approved mute. Remember a fresh misconfiguration only surfaces after the **next scan** — wait the scan window.

The `gcloud scc …` command is not recognized / flags differ

• **Likely cause:** the SCC command surface is **edition- and version-dependent**; the subcommand or flag differs in your tenant, or the SDK is outdated.

• **Resolve:** Run `gcloud components update`, then `gcloud scc --help` (and the subcommand's `--help`) to get the exact form for your edition. Adjust the step/tool argument accordingly — do not assume the form in this guide is identical to yours.

Notification config created but no messages on the topic

• **Likely cause:** the SCC **notification service account lacks** `roles/pubsub.publisher` on **Finding Export Topic – SecOps – v1**, or the config's `--filter` excludes the findings, or the service-account address is wrong.

• **Resolve:** Run `scripts/scc_scope_config_helper.py` to check the publisher binding; the SCC notification service account is **per-org** — verify its exact address in the Console before binding. Confirm the config filter actually matches a live finding (e.g. loosen to `state="ACTIVE"` to test). Pull the subscription to confirm: `gcloud pubsub subscriptions pull … --auto-ack`.

Messages on the topic but the SIEM isn't ingesting

• **Likely cause:** the **consumer subscription/connector** is misconfigured, the consumer service account lacks `roles/pubsub.subscriber`, or the **field mapping** doesn't match the SIEM's schema.

• **Resolve:** Confirm the subscription exists and the consumer identity has subscriber rights; re-check the ingestion contract (Requirements §5) and the SIEM connector against the **SIEM vendor's current docs**. Test with a manual pull to confirm the payload shape.

Permission denied creating configs / listing findings

• **Likely cause:** the caller lacks the SCC role (`roles/securitycenter.admin` to configure, finding-editor/viewer to triage) at the **Organization**.

• **Resolve:** Confirm with `gcloud organizations get-iam-policy ORG_ID`; have an Organization Administrator grant the correct least-privilege SCC role to the operating group (Requirements §2). Also confirm you are authenticated to the right account (`gcloud auth list`) and that ADC is set for the Python tools.

The finding queue is overwhelming (too much noise)

• **Likely cause:** a high-volume detector or unfiltered export is flooding the queue, so responders stop trusting it.

• **Resolve:** Identify the top categories with `scripts/scc_coverage_assessment.py`; fix the root cause where possible; **mute deliberately** (documented exception + owner + review date) via the setup tool; and tune the **export filter** so low-value findings don't reach the SIEM while remaining in SCC for

audit. Prefer filtering the export over disabling detectors.

A remediated finding stays ACTIVE

- **Likely cause:** the underlying resource wasn't actually fixed, the fix hasn't been re-scanned yet, or a different resource still violates the same detector.
- **Resolve:** Confirm the resource is genuinely compliant; wait for the next scan; check whether another resource under the same category is keeping the category "hot." SCC sets state to `INACTIVE` on the next scan after true remediation.

Triage/SLA report shows wrong owners or SLA math

- **Likely cause:** the **ownership matrix** or **severity→SLA table** in **Triage Runbook – SecOps – v1** is stale or mismatched to finding fields.
- **Resolve:** Update the runbook `owners` match rules (resource/category substrings) and `sla_hours`; re-run `scripts/scc_triage_sla_report.py`. Findings that map to `unassigned` mean no rule matched — add a rule.

`dry-run` looked fine but `--apply` failed

- **Likely cause:** a dependency the dry-run didn't execute (topic missing, API not enabled, IAM binding absent) blocks the real write.
- **Resolve:** Run `scripts/scc_scope_config_helper.py` to validate APIs, topic/subscription, and the publisher binding **before** `--apply`. Fix any FAIL, then re-run with `--apply` under change control.

Rollback needed after an org-wide change caused a noise spike or issue

- **Resolve:** Per the Implementation rollback — delete/disable **Finding Notification – Org – v1** to stop the export stream (findings stay in SCC), delete mute configs to reverse mutes, and re-enable any service you disabled. No finding data is lost by these config reversals; record each change so it reverts cleanly.

Best Practices

Durable practices for keeping Security Command Center (SCC) valuable — high signal, owned findings, and a closing loop — after the pilot.

Coverage & edition

- **Re-verify the edition periodically.** SCC editions, included services, and detectors change.

Re-run the coverage assessment (script #1) on a schedule and re-confirm what your edition includes against Google's current docs — don't assume last quarter's detector set still holds.

- **Keep the whole hierarchy in scope.** New folders/projects should inherit SCC coverage; audit for orphaned projects that fell outside scope.

Noise control (so responders trust the queue)

- **Mute deliberately, document every exception.** Every mute in **Posture Baseline – Corp – v1** has a written justification, an owner, and a review date. Un-audited mutes hide real risk.
- **Tune the export filter, not the detectors.** Prefer filtering what streams to the SIEM over disabling detectors — you keep the finding in SCC for audit while sparing the SIEM the noise.
- **Review high-volume categories first.** A single noisy detector can drown the queue; fix the root cause (e.g. a standard that generates thousands of low-value findings) rather than muting blindly.

Triage & ownership

- **Own by scope, not by person.** Route findings to **groups** (per the ownership matrix), so triage survives staffing changes.
- **SLA by severity, tracked automatically.** Run the triage/SLA report (script #3) on a cadence; surface at-risk and breached findings to owners before they age out.
- **Close the loop on remediation.** A finding isn't done until SCC shows it `INACTIVE` on rescan — track re-open rate; a finding that keeps returning signals a systemic gap, not a one-off.
- **Accept-risk is a decision, not a default.** If a Low finding won't be fixed, record an explicit risk acceptance (owner + expiry) rather than letting it linger unworked.

Notifications & export

- **One governed export path.** Keep **Finding Export Topic – SecOps – v1** in the SecOps project with least-privilege IAM; don't scatter ad-hoc notification configs across projects.
- **Least privilege for consumers.** The SIEM/consumer service account gets only `roles/pubsub.subscriber` on the subscription — never Owner, never an exported key committed to disk. Prefer workload identity / ADC.
- **Verify the SCC notification service-account binding** after any org change; a missing `roles/pubsub.publisher` binding silently stops the stream.

Security & safety

- **No secrets in automation.** The provided tools use ADC / the environment; never hardcode

credentials or commit service-account keys. Restrict who can create notification configs and mute rules (privileged, org-level).

- **Change control for org-wide actions.** Enabling/disabling services, org-wide mutes, and notification changes go through change control with the documented rollback ready.
- **Least-privilege triage roles.** Triagers edit finding state; only admins configure services and mutes.

Operating rhythm

- **Weekly triage review** of the SLA report; **monthly** posture-drift and mute review; **quarterly** edition/coverage re-verification. Trend finding volume, time-to-close, and re-open rate — a falling time-to-close and re-open rate is the proof the program is working.
- **Keep the runbook current.** As teams and services change, update **Triage Runbook – SecOps – v1** so ownership never goes stale.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

- **Solution Overview** — what SCC is, how posture/vulnerability/threat findings, notifications, export, and triage fit together; the reference design (named artifacts); benefits, limitations, and the pilot-first deployment model. Flags that **editions and detectors are version-dependent — verify your edition.**
- **Requirements Guide** — the solution-specific prerequisites (confirmed SCC edition, SCC IAM roles, required APIs, a SecOps project, a SIEM/notification destination, a triage operating model, a named pilot scope), each with how to *verify* and *implement*, building on the shared GCP global requirements.
- **Implementation Guide** — exact ordered steps: assess coverage, design the posture baseline and export topology, enable/tune services, create the notification config + export topic, apply the posture baseline and mutes, run a pilot triage cycle, then expand org-wide — with pass/fail gates and a documented rollback.
- **Testing & Validation** — nine procedures (coverage, notification delivery, SIEM ingestion, detector fires, mute behavior, remediation closure, triage/SLA report, least privilege, rollback), each with an explicit pass/fail signal.
- **Troubleshooting** — common failure modes (no findings, no notifications, SIEM not ingesting, noisy queue, permission errors) and their resolutions.
- **Best Practices** — coverage, noise control, triage/ownership, export hygiene, security, and the operating rhythm that keeps findings worked.

- **Executive Summary** — the outcome, approach, and why it is safe/governed.

Python tools (5)

All are `gcloud` / Cloud Client Library automation, **read-only or dry-run by default**, with no hardcoded credentials or committed keys (they use ADC / the environment). Run any with `--help`.

1. `scc_coverage_assessment.py` — *Assessment / discovery (read-only).* Inventories the

Organization's SCC state: active services, existing notification configs, and finding counts by severity/category — the baseline for the design. Outputs JSON.

1. `scc_notify_export_setup.py` — *Configuration helper (dry-run default).* Sets up the

notification/export plumbing — creates **Finding Notification – Org – v1** publishing to **Finding Export Topic – SecOps – v1**, and (optionally) applies posture **mute** rules. Prints a plan; only writes with `--apply`.

1. `scc_triage_sla_report.py` — *Validation / report generator (read-only).* Pulls active

findings, maps each to its owner and severity→SLA from **Triage Runbook – SecOps – v1**, computes due dates and at-risk/breached flags, and emits an HTML triage report.

1. `scc_posture_check.py` — *Policy / guardrail check (read-only).* Compares live posture

findings and mute state against **Posture Baseline – Corp – v1**, reporting drift (un-approved mutes, missing exceptions, unexpected active detectors). Exits non-zero on drift for CI.

1. `scc_scope_config_helper.py` — *Scope / config helper (read-only).* Resolves and validates the

SCC scope (org/folder/project), the SecOps project, and the export topic/subscription wiring — confirming IAM bindings (e.g. the SCC notification service account's publish rights) so the other tools have a correct, verified target.

Packaged artifacts

- `GCPSCC-Solution.pdf` — the compiled documentation set above.
- `GCPSCC-Scripts.zip` — the five Python tools.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

scc_coverage_assessment.py

scc_notify_export_setup.py

scc_posture_check.py

scc_scope_config_helper.py

scc_triage_sla_report.py